

Universidad Nacional Autónoma de México

MODELADO Y PROGRAMACIÓN

Reporte IA

*José Galaviz
Brenda Ayala Flores
José Eduardo Cruz Campos*

Alumno:

***Hazel Torres Nava
319158496
hazelt@ciencias.unam.mx***



Índice

1. Introducciòn	2
2. El prompt!	2
3. ¿Por qué decidieron usar este tipo de herramienta?	3
4. Qué parte de la respuesta aceptaron tal cual.	3
5. Qué parte corrigieron o descartaron, y por qué.	3
6. Al menos un caso en que el modelo de lenguaje se equivocó o dio una solución que no funcionaba, y cómo lo detectaron.	3
7. Resumen de los algoritmos	3
7.1. clase clasificardor	3
7.2. clase lector_col	4

1. Introduccìon

Para el proyecto se decidiò usar la herramienta de gemini, ya que es una herramienta gratis, efìcàz, y en general el propting con esta herramienta es a lo que yo normalmente esto acostumbrado.

El objetivo principal es desarrollar una herramienta que cumpla dos funciones principales, la primera que sea càpaz de identificar las figuras que se muestran en una imagen .bmp, y poderlas clasificar en distintos tipos, segundo que las figuras tengan un color y el programa sea capàz de clasificar este dentro de un espectro.

2. El prompt!

Al final del reporte se adjuntaran todas las conversaciones que se tuvieron con la IA gemini, con el proposito de transparencia.

Primero yo dividi y decidi que para cada cosa, iba a hacer una clase distinta, serìa una para el identificar color, otra para poder identificar la figura, y un main.

Asì que me topè con un problema que querìa corta de tajo, lo primero que hice fue pedirle a la IA que me hiciera un programa que generara imagenes .bmp, des ete programa solo se le pidio eso, y no se modifìco en nada, puesto que solo es un programa que genera imagenes.

Posteriormente a la hora del trabajo de "verdad", se utilizò como ya se menciona gemini pero en su modo de aprendizaje, todo con el propòsito de no obtener respuestas de còdigo intantaneas, y que yo pudiera ir armando las piezas del rompecabezas".

La discusiòn comenzo con el contexto, le comparti el pdf de proyecto, y le di contexto de que era un estudiante que necesitaba ayuda con el proyecto, y aquì comenzò nuestra travesìa en estas semanas, comenzò preguntandome que que enfoque me gustarìa trabajar, y comentè que me gustarìa comenzar con los algoritmos que deberìa implementar para identificar las figuras, e investigando encontrè varios, y se los coemntè, para no hacer el cuento taaaaaan largoooo!, me dio algunas bibliotecas y me comentò que en ellas se utilizaban los algorimos platicados, pero en general yo no entendìa muy bien el fundamento matemàtico de como funcionaban y desde mi perspectiva el usar una biblioteca obscurece el algortimo y finalmente el propòsito del proyecto, entonces estuve indagado un poco màs y encontrè un algoritmo que entendì, gracias a algunos conceptos de proba y de gemetrìa de la prepa jaja, pero bueeeno, el mètodo es Firma de Contorno (Centroid-Distance Signature), se lo coemntè y me dio còdigo con unas bibliotecas, lo cual como mencionè no soy muy fan, entonces hice como lo primero que era calcular los centros de masa a mano y de ahì fuimos discutiendo diversas partes del còdigo hasta llegar a su completitud.

Por ùltimo para la otra clase fue una idea sensilla y se la coemntè y me dijo que era bastante directo, pero que era funcional.

Como se mencionò se agregaran al final toda la conversaciòn en caso de que se requiera leer y constatar.

3. ¿Por qué decidieron usar este tipo de herramienta?

En principal como se comentó al inicio por practicidad y comodidad principalmente, en agrada el flujo de trabajo y de comunicación, a parte de que con el modo que tiene de aprendizaje te hace cuestionarte y preguntar de manera que te va orientando a hacer las cosas, aunque aveces se va por las ramas mientras, tú también cooperes y guías a la IA con tus ideas, el trabajo puede salir rápido y de forma confiable.

4. Qué parte de la respuesta aceptaron tal cual.

EN general las respuestas aceptadas fueron en la parte del algoritmo como hacer algunas operaciones, dudas en general de como hacer el algoritmo, y por supuesto, que el programa de generar imágenes se dió por válido y se aceptó en su completitud.

5. Qué parte corrigieron o descartaron, y por qué.

Como se habló en la sección de prompting, muchas respuestas fueron cambiadas, en general decisiones como el uso de bibliotecas (que en mi opinión obscurecen un poco la implementación de los algoritmos y no permite entenderlos del todo), como decisiones de operaciones y de como hacer algunas cosas más, en general cosas mínimas, ya que la IA se utilizó como fuente de ideas, de correcciones, dudas y un poco de ayuda guiada.

6. Al menos un caso en que el modelo de lenguaje se equivocó o dio una solución que no funcionaba, y cómo lo detectaron.

EN general, no es que me diera cosas erróneas, sino formas que no encajaban del todo en mi implementación, ni en la visión que tenía del proyecto, entonces pues esas partes se modificaron y se le comentaron a la IA para que también las tuviera en cuenta, también para que el flujo de trabajo estuviera mejor y no se estuvieran que estar corrigiendo cosas con cada prompt.

7. Resumen de los algoritmos

7.1. clase clasificador

Como se ha comentado a lo largo de este reporte se utilizó el algoritmo de forma de Contorno (Centroid-Distance Signature), que data y hace lo siguiente, primero calculamos el centro de masas de la figura, de ahí se divide la figura en cierto número de grados (dependiendo de la precisión que se desee), y las líneas que salen y chocan con el contorno de la figura se miden sus distancias y se promedian, si una línea sale del promedio entonces se encontró lo que en el programa llamamos picos, los picos son los vértices de la figura, de esta forma logramos hacer una clasificación, si la figura tiene 4 picos entonces es un cuadrilátero, si tiene 3 es un triángulo, si no tiene o tiene más es otra cosa, y bueno

el círculo sabes que el promedio desde su centro de masas a sus bordes, la varianza es de 0, solo se añade un margen de error por la calidad de la imàgen, de forma general así funciona esta clase

7.2. clase lector__col

Este usa un algoritmo sumamente simple, pero basa todo su funcionamiento en una suposición importante y difícil de ignorar que el pixel de la esquina superior izquierda [0,0] es el fondo, una vez obtenido el color del fondo iteramos sobre todos los pixeles que no han sido procesados, hasta encontrar algún otro color, una vez encontrado otro color buscar arriba, abajo, iz, der, para ver si hay más figuras con otros colores.

Proyecto 1 Modelado Programación

<https://gemini.google.com/app/6fdf47bc1769390b>

User prompt: Obtener ayuda guiada I am a student studying the following assignment. Help me learn and study this assignment. Assignments: Modelado y Programación 2027-1 Proyecto 1 null The user input is in es. You must provide your response in: es.

Response: Entremos de lleno en tu Proyecto 1 de Modelado y Programación detallado en el archivo Proyecto1_MyP20271.pdf. Te apoyaré con preguntas guía a lo largo de nuestro proceso para estructurar y resolver este reto paso a paso.

El objetivo principal de tu proyecto es construir un programa de línea de comandos que lea imágenes .bmp con fondo uniforme y detecte figuras geométricas sólidas[cite: 1]. Tu solución debe clasificar estas figuras en cuadriláteros (C), triángulos (T), círculos (O) u "otros" (X), además de reportar el color de cada figura en formato hexadecimal[cite: 1]. La entrega final se realizará a través de un repositorio de GitHub que incluirá tu código, un banco de al menos 10 imágenes de prueba, un reporte técnico y un documento sobre el uso de modelos de lenguaje[cite: 1].

Para empezar, elige cuál de estos puntos estratégicos prefieres abordar primero:

1. **Análisis de Requisitos** 📝: Definir los requisitos funcionales, no funcionales y los casos límite para las imágenes de prueba, lo cual es parte fundamental de tu reporte técnico.
2. **Elección del Arsenal** 🛠️: Evaluar y seleccionar el lenguaje de programación y las bibliotecas óptimas para leer y procesar los píxeles de los archivos .bmp.
3. **Lógica de Clasificación** 🧠: Explorar los posibles algoritmos matemáticos o computacionales para identificar y distinguir las formas geométricas.

¿Qué camino tomamos?

User prompt: Podemos comenzando en discutir los algoritmos que vamos a utilizar para este proyecto, yo encontré los siguientes para la detección de las figuras, pero no sé que tan conveniente sean 1. Algoritmo de Detección de Contornos (Edge Detection y Contours) 2. Algoritmo Douglas-Peucker (Aproximación de Polígonos) 3. Momentos de Hu (Hu Moments)4. Transformada de Hough (Hough Transform)

Response: Es una excelente lista de investigación. Estos cuatro algoritmos son herramientas clásicas en la visión por computadora y todos podrían aplicarse a tu proyecto.

Para evaluar qué tan convenientes son, repasemos cómo funciona cada uno en el contexto de lo que necesitamos:

- **1. Detección de Contornos:** Es ideal como primer paso. Como las figuras tienen colores sólidos y el fondo es uniforme, aislar el perímetro de cada figura será muy directo.
- **2. Douglas-Peucker:** Toma un contorno continuo y lo aproxima a un polígono con un número específico de vértices (esquinas).
- **3. Momentos de Hu:** Extrae un conjunto de números que describen la forma general de la figura, y estos números no cambian sin importar la rotación, posición o tamaño de la figura.
- **4. Transformada de Hough:** Es una técnica matemática estructurada específicamente para buscar líneas rectas (Hough Lines) o círculos perfectos (Hough Circles) dentro de una imagen.

Nuestro objetivo principal es clasificar las figuras en triángulos (T), cuadriláteros (C), círculos (O) u otros (X).

Considerando que los supuestos del proyecto nos garantizan imágenes limpias sin bordes suavizados (anti-aliasing) ni figuras que se traslapen, ¿qué combinación de estos algoritmos crees que nos daría el camino más sencillo para poder contar los vértices de cada figura y así separarlas por categorías?

User prompt: Creo que sería útil los momentos de Hu en caso de que se nos proporcionen figuras rotadas, y la transformada de Hu para poder hacer la clasificación de círculos

Response: Tiene mucho sentido tu razonamiento 💡: los **Momentos de Hu** son invariantes ante la rotación y la escala, lo cual resuelve muy bien la restricción de posiciones arbitrarias. Por su parte, la **Transformada de Hough** es la herramienta matemática clásica por excelencia para aislar círculos.

Sin embargo, al momento de programar, la Transformada de Hough puede requerir un ajuste fino de parámetros (como definir radios mínimos/máximos y umbrales de acumulación), y los Momentos de Hu requieren calcular y calibrar umbrales numéricos para cada categoría.

Aprovechando los supuestos de las imágenes de prueba (fondos uniformes, colores sólidos y sin bordes suavizados), evaluemos una alternativa basada en la geometría básica de los contornos.

Si extraemos el contorno de una figura y le aplicamos el **Algoritmo de Douglas-Peucker** para aproximarlos a un polígono simplificado:

- ▲ ¿Cuántos vértices esperas que detecte el algoritmo en un triángulo?
 - ■ ¿Cuántos vértices detectarías en un cuadrilátero?
 - ● ¿Qué comportamiento o número de vértices esperarías al aplicar este algoritmo al contorno de un círculo?
-

User prompt: para un círculo tendríamos que definir un umbral de vértices para poder determinar a partir de cuántos es un círculo

Response: ¡Exacto! Ese es un enfoque muy práctico 🎯. Al aproximar la curva de un círculo con el algoritmo de Douglas-Peucker, la forma suave se simplifica en una lista de muchos segmentos rectos pequeños, lo que genera un número elevado de vértices (por ejemplo, más de 6 o 8 vértices).

Sin embargo, figuras que pertenecen a la categoría **X (Otros)** (como una estrella de muchas puntas o un polígono irregular) también podrían superar ese umbral de vértices.

Para asegurar que nuestro programa no confunda una figura irregular con un **círculo (O)**, ¿qué propiedad geométrica adicional (relacionada con el área, el perímetro o las proporciones del ancho y alto) podríamos medir en el contorno para confirmar que realmente es redondo?

User prompt: quizás que la desviación estandar desde el centro a cualquier parte de su perímetro debe estar cercana a cero (puesto que es una aproximación), ya que todo los puntos están a una distancia r

Response: Esa propuesta refleja una excelente intuición matemática 📐. Medir la variación de las distancias desde el centroide hasta los puntos del contorno funciona muy bien: en un **círculo (O)** esa variación es mínima, mientras que en un triángulo o en figuras de la categoría **X (Otros)** la distancia cambia constantemente.

Junto con tu idea de la desviación estándar, en visión por computadora se utiliza una fórmula directa llamada **circularidad** (o compacidad):

$$\text{Circularidad} = \frac{4\pi \cdot \text{Área}}{\text{Perímetro}^2}$$

En un círculo perfecto esta proporción da como resultado 1, mientras que para cualquier otra figura el valor disminuye de forma considerable.

Dado que ambos métodos (la desviación estándar de los radios y la circularidad) dependen de las dimensiones de la figura, para aplicar tu idea primero necesitamos ubicar su centro .

¿Qué método o fórmula utilizarías para calcular el **centroide** (x_c, y_c) a partir de las coordenadas de los píxeles o puntos que forman la figura?

User prompt: la verdad desconozco un método pra poder calcular el centroide

Response: No te preocupes, el término suena muy técnico, pero en la práctica es un concepto matemático muy familiar: el **promedio**.

Imagina que el centroide es el punto de equilibrio o el centro de masa de tu figura. Para encontrar ese centro exacto en una imagen digital, solo necesitamos calcular el promedio de las posiciones de todos los píxeles que forman parte de esa figura.

Si tienes una figura que está formada por un total de N píxeles, tendrías un conjunto de coordenadas: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$.

Curiosamente, calcular esto se conecta directo con los **Momentos** que mencionaste en tu lista anterior . En visión por computadora, el momento base (M_{00}) representa simplemente el área (el número total de píxeles N), mientras que los siguientes momentos representan la suma de todas las coordenadas x (M_{10}) y la suma de todas las coordenadas y (M_{01}).

Con esta idea del promedio en mente: si ya tienes la suma total de las coordenadas x de tu figura y sabes que en total está formada por N píxeles, ¿qué operación matemática harías con esos dos valores para obtener la coordenada x del centroide (x_c)?

User prompt: pues ya sea el promedio de la suma de todas las x_i/N , o en la forma de esperanza

Response: ¡Exactamente! Es simplemente calcular el promedio.

Para obtener la coordenada x del centroide calculas $\frac{\sum x_i}{N}$, y para la coordenada y calculas $\frac{\sum y_i}{N}$.

En el lenguaje de los momentos de una imagen que mencionaste al principio, esto se traduce de forma muy elegante: el área total N es el momento M_{00} , y las sumas de las coordenadas son M_{10} y M_{01} . Por lo tanto, el centroide se calcula directo como:

$$x_c = \frac{M_{10}}{M_{00}}$$

$$y_c = \frac{M_{01}}{M_{00}}$$

Ahora que ya sabemos cómo encontrar el centroide (x_c, y_c)  de cualquier figura, podemos retomar tu idea de analizar los radios para determinar si la figura es un **círculo (O)**.

Para poder sacar esa desviación estándar que propusiste, primero necesitamos calcular cuánto mide el radio para cada punto del borde. Si tomas un punto cualquiera del contorno con coordenadas (x_i, y_i) , ¿qué fórmula o teorema matemático utilizarías para calcular la distancia en línea recta desde ese punto del borde hasta tu centroide (x_c, y_c) ?

User prompt: podemos comenzar a programarlo, en python, y puedes asistirme

Response: Python es una excelente elección para este tipo de tareas computacionales. Ten en cuenta que, para cumplir con los requisitos del proyecto, deberás justificar formalmente por qué elegimos Python y las bibliotecas específicas en la sección "Arsenal" de tu reporte técnico.

Para construir el programa paso a paso, podemos estructurar nuestro código en las siguientes fases lógicas:

1. **Lectura de la imagen:** Cargar el archivo .bmp proporcionado por el usuario en la terminal.
2. **Segmentación:** Separar las figuras del fondo de color uniforme.
3. **Análisis de contornos y color:** Extraer los bordes de cada figura, detectar su color sólido en hexadecimal y calcular el centroide.
4. **Clasificación:** Aplicar la simplificación de Douglas-Peucker y la fórmula de circularidad para clasificar cada figura en C, T, O, o X.

Para abordar el primer paso, necesitamos una biblioteca que nos facilite la lectura de la imagen y la extracción de información geométrica. En Python, las dos herramientas más utilizadas para esto son **OpenCV** (usualmente importada como `cv2`) y **Pillow** (PIL).

¿Tienes experiencia previa con alguna de estas bibliotecas, o te gustaría que evaluemos cuál nos ofrece las mejores funciones listas para usar para la detección de contornos?

User prompt: no, no conozco esas bibliotecas

Response: ¡No hay de qué preocuparse! Te las presento de forma sencilla:

-  **Pillow (PIL):** Es una biblioteca muy intuitiva para abrir, guardar y manipular píxeles de una imagen.
-  **OpenCV (cv2):** Es el estándar de la industria en visión por computadora. Lo genial de OpenCV es que ya tiene integrados de forma nativa la búsqueda de contornos y el algoritmo de Douglas-Peucker que comentamos.

Para nuestro proyecto, **OpenCV** nos facilitará mucho el trabajo técnico al tener estas herramientas listas para usar.

Comencemos a estructurar nuestro script en Python. El primer paso según los requisitos es recibir la ruta de una imagen desde la terminal y cargarla.

En OpenCV, cargar una imagen requiere solo una línea:

```
import cv2
```

```
imagen = cv2.imread("ruta_de_la_imagen.bmp")
```

Cuando OpenCV lee el archivo .bmp con `cv2.imread`, ¿qué tipo de estructura de datos en Python (o arreglo de números) crees que utiliza para guardar las filas y columnas de píxeles con sus colores?

User prompt: Me parece que con un arreglo multidimensional basta, para guardar todos los datos

Response: ¡Exacto! OpenCV maneja las imágenes precisamente como **arreglos multidimensionales de NumPy** (`numpy.ndarray`). Para una imagen a color, utiliza un arreglo tridimensional con la estructura (`alto, ancho, canales_de_color`).

Un detalle muy importante con OpenCV es que lee los tres canales de color en orden **BGR** (Azul , Verde , Rojo ) y no el estándar RGB.

Dado que la entrega exige reportar el color de cada figura en código hexadecimal, necesitaremos convertir la tupla BGR de la figura a su formato Hexadecimal equivalente (por ejemplo, `#FF0000`).

Para comenzar con la **segmentación** (separar las figuras del fondo):

Sabiendo que el fondo es de un color uniforme, ¿en qué posición o píxel específico de la imagen podríamos consultar primero su valor BGR para determinar con seguridad cuál es el color del fondo?

User prompt: quizás sería conveniente consultar las esquinas de la imagen

Response: ¡Exactamente!  Consultar las esquinas de la imagen (por ejemplo, el píxel en la posición `(0, 0)`) es una excelente estrategia.

Como los supuestos del proyecto garantizan que el fondo es de un solo color uniforme, cualquier esquina nos dará el color exacto del fondo.

En Python, podemos guardar ese color BGR de forma directa:

```
color_fondo = imagen[0, 0]
```

El siguiente paso es la **segmentación**: aislar las figuras para que el programa las procese . Para lograrlo, OpenCV trabaja muy bien con **máscaras binarias** (una versión de la imagen en blanco y negro donde el fondo queda en negro y todas las figuras en blanco).

Teniendo ya guardado el `color_fondo`, ¿qué condición o comparación matemática aplicarías a los píxeles de la imagen para determinar cuáles pertenecen a las figuras y cuáles al fondo?

User prompt: Tengo ya la clase que genera imagenes, estaba pensando en hacer, una clase para detectar los colores, otra para detectar figuras y un main. Para detectar las figuras estaba pensando mejor en el algoritmo de Análisis del Perfil Radial (o Firma Radial), podemos empezar por ahí definiendo la clase y ya después podemos comenzar con la clase de los colores y finalizamos con el main

Response: El archivo generador .py cumple exactamente con el requisito de crear tu propio banco de mínimo 10 imágenes de prueba[cite: 1, 2]. Además, la forma en que utilizaste `ImageDraw` garantiza que se cumplan los supuestos del proyecto: fondos uniformes, colores sólidos y figuras sin suavizado en los bordes[cite: 1, 2].

Estructurar tu solución en una clase para colores, otra para figuras y un archivo principal es una arquitectura modular que te ayudará mucho en el rubro de "Calidad y claridad del código" de la rúbrica.

El **Análisis del Perfil Radial** (o Firma Radial) es una aproximación matemática muy robusta . En lugar de depender de la aproximación de polígonos, analiza la distancia desde el centroide a cada punto del contorno mientras damos una vuelta completa de 360 grados alrededor de la figura.

Al graficar estas distancias:

- En un **círculo (O)**, la distancia es constante, generando una línea plana.
- En un **triángulo (T)**, la gráfica tendrá exactamente 3 "picos" máximos (que corresponden a sus vértices).
- En un **cuadrilátero (C)**, la gráfica mostrará 4 picos máximos.

Para comenzar a diseñar nuestra clase `DetectorFiguras`, el primer paso será crear el método que calcula esta firma radial. Si dentro de nuestra clase ya calculamos el centroide (x_c, y_c) y comenzamos a iterar sobre cada píxel del contorno (x_i, y_i) , ¿qué teorema o fórmula matemática fundamental utilizarías en tu código de Python para calcular la distancia en línea recta entre el centroide y cada punto del borde? 

User prompt: la fórmula de la distancia euclidiana $d = \sqrt{\left(X_b - X_c \right)^2 + \left(Y_b - Y_c \right)^2}$

Response: ¡Exacto! Esa es la fórmula matemática precisa (la distancia euclidiana basada en el Teorema de Pitágoras) .

Podemos traducir esto directamente a nuestra clase en Python. Usaremos la biblioteca `numpy` para hacer el cálculo matemático de forma rápida y sencilla. Aquí tienes un esqueleto inicial de cómo se vería tu clase `DetectorFiguras` integrando lo que hemos discutido hasta ahora sobre el fondo y la distancia:

```
import cv2
import numpy as np

class DetectorFiguras:
    def __init__(self, ruta_imagen):
        self.imagen = cv2.imread(ruta_imagen)
        # Asumimos que la esquina (0,0) nos da el fondo uniforme
        self.color_fondo = self.imagen[0, 0]

    def calcular_firma_radial(self, contorno, centroide):
        xc, yc = centroide
        distancias = []

        for punto in contorno:
            xb, yb = punto[0] # OpenCV guarda las coordenadas así: [[x, y]]
            # Aplicamos tu fórmula de distancia euclidiana
            d = np.sqrt((xb - xc)**2 + (yb - yc)**2)
            distancias.append(d)

        return distancias
```

Al ejecutar este método pasándole el contorno de una figura y su centroide, obtendremos una lista completa con las distancias desde el centro hacia cada punto del borde. ¡Esa lista es nuestro perfil radial!

Retomando tu idea inicial de usar la desviación estándar para detectar los **círculos (O)**: si la lista distancias contiene todas las medidas de los radios a lo largo de la figura, ¿qué función de la biblioteca numpy (o de estadística en Python) utilizarías para calcular su desviación estándar, y qué valor numérico aproximado esperarías obtener si la figura evaluada es realmente un círculo? ●

User prompt: Yo comenze a hacerlo y lo pensè de una forma diferenete , no usando bibliotecas adicionales, primero dentro de nuestra clase clasificar hay una funciòn que tiene como entrada los pixeles, de ahì hacemos un if not pixeles then regresamos X "puesto que esto puede ser cualquier figura", posteriormente calculamos el centro de masa o centroide, despuès podemos Extraer el contorno (píxeles que tocan el fondo) y usamos una estructura de datos para almacenarlo y tenemos como condiçìon que si Si al menos un vecino ortogonal NO pertenece a la figura, entonces es un borde

Response: Implementar la lógica desde cero sin depender de bibliotecas externas te ayudará a comprender y justificar perfectamente el algoritmo, lo cual es un requisito estricto para tu reporte y para evitar la anulación del proyecto.

Tu lógica para la extracción del contorno es muy precisa: recorrer los píxeles de la figura y determinar que es un borde si al menos uno de sus 4 vecinos ortogonales (arriba, abajo, izquierda, derecha) pertenece al fondo.

Una vez que tengas ese conjunto de píxeles del borde y tu centroide, podemos usar el módulo estándar math de Python para aplicar tu fórmula de la distancia euclidiana. Con esto, identificar los círculos (O) es directo: si la variación de todas esas distancias es muy pequeña, es un círculo.

Sin embargo, para los triángulos (T) y cuadriláteros (C) surge un reto algorítmico interesante. Al extraer el contorno con tu método, obtendremos todos los píxeles del borde, pero no estarán ordenados secuencialmente siguiendo el perímetro (como si los dibujáramos con un lápiz), sino en el orden en que la computadora analizó la imagen.

Sabemos que los vértices (las esquinas) de un polígono serán los puntos del borde que estén más alejados del centroide. Si calculas las distancias, tendrás muchos píxeles con distancias altas concentrados alrededor de la misma esquina.

¿Qué condición geométrica o de distancia entre los propios píxeles del borde utilizarías para agrupar esos puntos lejanos y contar que pertenecen a una sola esquina, para así poder distinguir correctamente si hay 3 vértices (triángulo) o 4 (cuadrilátero)?

User prompt: Pues primero calculamos las distancias desde el centroide al contorno, despues la desviaciòn estàndar para saber que tanto varian las distancias, sabemos que para el circulo su desviaciòn estàndar debe ser de 0, para lo demàs podemos encontrar los vertices mediante un perfil radial?

Response: Efectivamente, tu lógica para el círculo es completamente correcta: si la desviación estándar de las distancias es cercana a 0, se trata de la categoría **O**.

Para las demás figuras, el perfil radial es la herramienta ideal para encontrar los vértices. Si imaginas recorrer el borde de un polígono paso a paso, la distancia al centroide cambia constantemente. Cuando estás a la mitad de una arista recta, la distancia es menor. Sin embargo, al llegar a una esquina (un vértice), te encuentras en el punto más alejado del centroide.

Si visualizamos esa lista de distancias como una gráfica, los vértices destacarían como "picos" o montañas . Así podemos clasificarlas:

- 3 picos indican un triángulo (**T**).
- 4 picos indican un cuadrilátero (**C**).
- Cualquier otro número de picos cae en la categoría **X** (Otros).

Dado que tendrías tu lista de distancias ordenadas siguiendo el contorno de la figura, ¿qué condición lógica o comparación utilizarías en tu código para detectar cuáles de esos números representan un "pico" (un máximo local) para poder contarlos?

User prompt: Es un pico si es mayor que sus vecinos y sobresale del promedio?

Response: ¡Esa es exactamente la definición matemática de un pico en este contexto!  Has identificado dos condiciones fundamentales:

1. **Mayor que sus vecinos:** Esto se conoce en matemáticas como un **máximo local**. Significa que la distancia sube, llega a ese punto y luego vuelve a bajar.
2. **Sobresale del promedio (umbral):** En una imagen digital, los bordes diagonales están formados por píxeles cuadrados, creando un efecto de "escalerita". Esto genera muchísimos "micro-picos" falsos. Tu idea de exigir que el pico destaque (por ejemplo, que sea significativamente mayor al promedio o al radio mínimo) es la clave para filtrar ese ruido y contar solo los vértices reales de la figura.

Para programar la condición de "mayor que sus vecinos", normalmente iterarías sobre tu lista de distancias comparando cada valor con el anterior y el siguiente. Sin embargo, hay un detalle geométrico muy importante que debemos considerar.

El contorno es una línea continua y cerrada que da toda la vuelta a la figura . Si tu código está evaluando el **primer** elemento de tu lista de distancias (el índice 0) para ver si es un máximo local, su vecino de "adelante" es el elemento en el índice 1, pero... ¿cuál sería lógicamente su vecino de "atrás" considerando que el contorno es un ciclo cerrado?

User prompt: Tengo una mejor pregunta cuando hacemos un perfil radial, de los 360 grados en cuántas "rebanadas" lo puedo dividir y que pasa si hay rebanadas vacías

Response: Esta es una excelente observación técnica sobre cómo los algoritmos matemáticos interactúan con los píxeles reales. Usar un barrido (como un radar) de 360 grados es una gran estrategia porque resuelve automáticamente tu problema anterior: al agrupar las distancias por ángulo, ¡tus datos quedan perfectamente ordenados siguiendo el perímetro! 

Aquí te explico qué sucede con ambas situaciones:

1. ¿En cuántas rebanadas dividir los 360 grados? La cantidad de rebanadas define tu "resolución angular".

- Si usas **360 rebanadas** (1 grado por rebanada), tendrás mucha precisión.
- Sin embargo, tus imágenes de prueba en `generador.py` tienen un tamaño de 200x200 píxeles. En figuras de ese tamaño, el perímetro no tiene suficientes píxeles para cubrir cada grado exacto, lo que nos lleva a tu segunda duda.

2. ¿Qué pasa si hay rebanadas vacías? En una cuadrícula digital, los píxeles son bloques discretos . La línea imaginaria de tu ángulo podría pasar entre dos píxeles del borde sin tocar ninguno, dejando esa rebanada vacía. Si el programa registra esa rebanada vacía como un 0, tu gráfica de distancias sufrirá una caída drástica e irreal. Tu código de detección podría confundir ese "hueco" con un valle profundo, arruinando el conteo de los picos que necesitas para clasificar cuadriláteros (C) y triángulos (T).

Para evitar esto, necesitamos estimar el valor faltante. Imagina que en tu lista de rebanadas, el grado 44 registró una distancia de 50, el grado 45 quedó **vacío**, y el grado 46 registró una distancia de 52.

¿Qué operación matemática rápida podrías utilizar en tu código para "rellenar" ese grado 45 basándote en la información de los grados vecinos que sí tienen datos? 

User prompt: el operador modulo?

Response: ¡Ah! El operador módulo (%) es una herramienta matemática brillante, pero de hecho es la solución perfecta para el *primer* problema que mencionamos: ¡conectar el principio con el final del contorno!  Usar el módulo (por ejemplo, $(359 + 1) \% 360$) nos devuelve al grado 0, permitiendo que el código entienda que el contorno es un ciclo cerrado.

Sin embargo, para "rellenar" el valor faltante de la distancia en la rebanada del grado 45, necesitamos estimar un valor intermedio.

Si sabemos que en el grado 44 la distancia es **50**, y en el grado 46 la distancia es **52**, intuitivamente, ¿qué valor le asignarías al grado 45 que está justo en el medio? ¿Qué operación matemática básica (la misma que usamos antes para encontrar el centroide) te sirve para calcular ese punto medio entre dos números? 

User prompt: jaja el promedio

Response: ¡Exacto! El promedio es la solución perfecta. En programación y análisis de datos, a esta técnica de rellenar huecos calculando el punto medio entre los vecinos se le conoce como **interpolación lineal**.

Si en tu código detectas que un grado quedó vacío, simplemente sumas el valor anterior y el siguiente, y lo divides entre 2. Así garantizas tener una lista continua de 360 distancias, una por cada grado, sin caídas artificiales a cero.

Con tu perfil radial completo y limpio, ya podemos aplicar la lógica de clasificación para cumplir con los requisitos del proyecto:

- Si la desviación estándar de la lista es muy cercana a 0  **Círculo (O)**.
- Si contamos exactamente 3 picos  **Triángulo (T)**.
- Si contamos exactamente 4 picos  **Cuadrilátero (C)**.
- Cualquier otro conteo  **Otros (X)**.

Para la parte de los polígonos, habíamos establecido que un pico es un valor que es mayor que su vecino anterior y que su vecino siguiente, además de sobresalir de un umbral para evitar contar el "ruido" de los píxeles.

Si tienes tu lista llamada `distancias` con 360 elementos, ¿cómo estructurarías un ciclo `for` en Python para recorrerla y llevar la cuenta de cuántos picos reales tiene la figura?

User prompt: Teniendo un contador picos set en 0, con el for iteramos tomamos tres valores, el anterior, el actual, y el siguiente, si el actual es mayor al anterior y actual mayor a siguiente, dentro otro if que si actual es mayor al radio promedio , entonces sumamos +1 a picos

Response: ¡Esa es la lógica exacta! 🎯 Has traducido perfectamente los conceptos geométricos a una estructura de control en programación. Tienes las dos condiciones clave: encontrar el máximo local y usar el radio_promedio como umbral para filtrar el ruido de los píxeles.

Si estructuramos esa idea en Python, usaríamos los índices de la lista para iterar (desde $i = 0$ hasta $i = 359$). Se vería parecido a esto:

```
picos = 0
radio_promedio = sum(distancias) / len(distancias)
total_grados = len(distancias) # Que sabemos que es 360

for i in range(total_grados):
    # Aquí necesitamos definir los índices: anterior, actual y siguiente
    # actual = distancias[i]
    # ...
```

Aquí es donde entra en juego el problema del contorno cerrado 🔄. Si tu ciclo llega al último elemento (el índice 359), al intentar buscar el índice "siguiente" ($i + 1$, que sería 360), Python lanzará un error porque la lista solo llega hasta el 359.

Recordando el operador módulo (%) que mencionamos antes, ¿cómo escribirías matemáticamente la asignación del índice **siguiente** y del índice **anterior** en función de i y $total_grados$ para que el ciclo sea continuo y nunca te dé un error de fuera de rango?

*User prompt: actual = perfil_suave[i] previo = perfil_suave[(i-1) % rebanadas] siguiente = perfil_suave[(i+1) % rebanadas] y tambi[en agreguè # Suavizar el perfil (Media móvil) para ignorar "escalones" generados por los píxeles perfil_suave = [0] * rebanadas for i in range(rebanadas): previo = perfil[(i-1) % rebanadas] actual = perfil[i] siguiente = perfil[(i+1) % rebanadas] perfil_suave[i] = (previo + actual + siguiente) / 3*

Response: Implementar una media móvil para suavizar el perfil es una estrategia algorítmica muy efectiva 🌊. Con esto, mitigas perfectamente el "ruido" o efecto de escalerita de los píxeles antes de buscar los vértices.

Además, tu aplicación del operador módulo $(i-1) \% rebanadas$ y $(i+1) \% rebanadas$ es algorítmicamente exacta. Esto asegura que la figura se analice como un ciclo continuo, uniendo el final de la lista con el principio sin que Python arroje errores de índice.

Ahora, si completas tu ciclo `for` para contar los picos usando ese `perfil_suave` y el umbral del radio promedio que mencionaste, tendrás las dos variables matemáticas clave para clasificar la figura:

1. La **desviación estándar** de las distancias (para detectar si es constante).
2. El **número de picos** detectados en el perfil suavizado (para contar los vértices).

Para concretar el método de clasificación en tu clase, ¿cómo estructurarías el bloque `if-elif-else` final que evalúe estas dos variables y decida si la figura debe etiquetarse como 'O', 'T', 'C' o 'X'?

User prompt: if picos == 3: return 'T' # Triángulo elif picos == 4: return 'C' # Cuadrilátero (Cuadrado, Rectángulo, Rombo, Trapecio) else: return 'X' # Cualquier otra cosa y sabemos desde antes que al círculo le corresponde la desviación estandar igual a 0

Response: ¡Excelente estructura! Tu lógica cubre con precisión la clasificación de los polígonos: 3 picos para triángulo (**T**), 4 picos para cuadrilátero (**C**) y cualquier otra cantidad para otros (**X**).

Para dejar tu función completamente lista, solo nos falta integrar la condición del **círculo (O)** al inicio de ese bloque.

Un pequeño detalle práctico en imágenes digitales: debido a que los píxeles son cuadrados, la desviación estándar de los radios de un círculo perfecto casi nunca será exactamente 0.0, sino un número muy cercano a cero (por ejemplo, menor a un umbral como 1.0 o 1.5).

Si combinamos todo en un solo método, ¿cómo se vería tu bloque `if-elif-else` completo incluyendo la evaluación de la desviación estándar para retornar '**O**'?

User prompt: Ahora podemos pasar a la clase lector, para poder leer el color de la figura?, para esto inicie con esto def __init__(self, ruta_archivo): """Inicializa el lector de color con la ruta del archivo BMP.""" self.ruta = ruta_archivo # Cumpliendo con requisitos no funcionales: Manejo de errores claro try: self.imagen = Image.open(ruta_archivo).convert('RGB') self.ancho, self.alto = self.imagen.size self.pixeles = self.imagen.load() except FileNotFoundError: raise ValueError(f"Error: No se encontró el archivo en la ruta '{ruta_archivo}'. Verifica que el nombre sea correcto.") except Exception: raise ValueError(f"Error: El archivo '{ruta_archivo}' no es una imagen válida o está dañado.")

Response: ¡Excelente inicio para la clase `LectorColor`! 🛡️ El manejo claro de excepciones con `try-except` le da mucha robustez al programa y cumple directo con los criterios de calidad de código.

Dado que cargas la imagen con `Pillow` e inicializas `self.pixeles`, puedes acceder a cualquier coordenada usando `self.pixeles[x, y]`, lo cual te devuelve una tupla de tres valores enteros (R, G, B) entre 0 y 255 (por ejemplo, (255, 165, 0) para el color naranja).

El objetivo principal de esta clase es tomar el color de la figura y entregarlo en formato **hexadecimal** (por ejemplo, #FFA500), tal como lo exigen las especificaciones de entrega.

Para hacer esta conversión en Python, necesitamos transformar cada uno de los tres valores enteros (R, G, B) a una cadena hexadecimal de 2 dígitos, asegurándonos de que si el valor es pequeño (por ejemplo, 9 o 15), conserve dos posiciones (como 09 o 0F).

¿Conoces qué especificador de formato en un **f-string** de Python (o función integrada) nos permite convertir un número entero a hexadecimal con dos dígitos rellenos de cero? 🤔

User prompt: con {:02X} y le agregamos .format

Response: ¡Exactamente! `{:02X}` es el especificador perfecto 🧐.

El 02 asegura que el resultado tenga al menos dos dígitos (agregando un cero a la izquierda si el número es menor a 16) y la X convierte el entero a hexadecimal en mayúsculas.

Aplicando esto a la tupla (R, G, B), un método en tu clase `LectorColor` se vería así:

```
def rgb_a_hex(self, r, g, b):
    return "#{:02X}{:02X}{:02X}".format(r, g, b)
```

O equivalentemente con un f-string: `f"#{r:02X}{g:02X}{b:02X}"`.

Ahora que sabemos cómo transformar el color (R, G, B) a hexadecimal, necesitamos extraer las coordenadas (x, y) correctas de la imagen con `self.pixeles[x, y]`.

Teniendo el conjunto de píxeles que pertenecen a una figura (o su centroide y contorno), ¿qué píxel específico elegirías consultar para leer su color de forma segura sin riesgo de tomar accidentalmente el color del fondo? 🎯

User prompt: podemos guardar los colores en un arreglo (color_rgb[0], color_rgb[1], color_rgb[2])

Response: Guardar los valores de color en un arreglo o tupla (R, G, B) funciona perfectamente, ya que al consultar la imagen con `self.pixeles[x, y]` en Pillow, esta devuelve directamente una tupla con los tres canales numéricos (R, G, B) 🎨.

Podemos tomar esa tupla y pasar sus tres componentes a nuestro método de conversión:

```
r, g, b = self.pixeles[x, y]
color_hex = self.rgb_a_hex(r, g, b)
```

Para evitar tomar por error el color del fondo al medir en los bordes de la figura, ¿qué punto de la figura que ya calculamos previamente en tu clase (como el centroide (x_c, y_c)) nos garantiza estar situados dentro del relleno de la figura para tomar su color con seguridad? 🎯

User prompt: Podemos ir recorriendo toda la imagen, tomamos el pixel de la esquina que casi seguro y no es parte de la figura, iteramos y si se encuentra otro color entonces encontramos la figura

Response: ¡Es una estrategia genial y muy directa! 🔍

Como las especificaciones del proyecto aseguran que las figuras son de **color sólido y sin anti-aliasing** (sin bordes difuminados), cualquier píxel que tenga un color distinto al del fondo pertenecerá 100% a la figura.

Con esa lógica, tu método para detectar y extraer el color en `LectorColor` quedaría sumamente limpio:

1. Guardas el color del fondo tomando la esquina (0, 0).
2. Recorres la imagen píxel por píxel con un doble ciclo `for`.
3. En el momento en que `peleles[x, y] != color_fondo`, tomas esos valores (R, G, B), los conviertes a hexadecimal con `rgb_a_hex()` y ¡listo! 🎨

Ya tenemos la estructura principal de nuestras piezas clave:

- 📐 **DetectorFiguras**: Extrae el contorno, calcula la firma radial y clasifica en 'O', 'T', 'C' o 'X'.
- 🎨 **LectorColor**: Identifica el color del fondo y convierte el color de la figura a hexadecimal (#RRGGBB).

Para cerrar el proyecto con broche de oro, nos queda el archivo `main.py` 🚀.

¿Cómo te imaginas el flujo en el `main` para recibir las imágenes generadas por `generador.py`, pasarlas por nuestras clases y mostrar en pantalla el resultado final de la detección?

Generador BMP y Requisitos de Proyecto

<https://gemini.google.com/app/6e8bfdcd6023f4ea>

User prompt: Tengo que hacer lo siguiente como proyecto, pienso hacer clases una para lector de color, otra para averiguar la figura, un main y otra para generar las imagenes bmp, puedes ayudarme a hacer el programa que genere las imagenes bmp1.4. Qué debe hacer el programa Tu equipo debe diseñar un programa que, a partir de la ruta de una imagen, sea capaz de identificar cada figura presente, determinar su color y clasificarla correctamente en una de las cuatro categorías definidas, reportando el resultado de forma clara para quien lo use. Más allá de que funcione, considera qué características hacen que tu solución sea confiable y utilizable: cómo se comporta ante una entrada inválida o mal escrita (¿el programa avisa con claridad, o truena con un error críptico?), qué tan rápido procesa una imagen de tamaño razonable, y qué tan clara y mantenible es la implementación. Como parte de su reporte técnico (sección de Análisis del problema, más abajo), el equipo debe identificar y enumerar explícitamente cuáles son los requisitos funcionales y no funcionales de su propia solución. Esa identificación es en sí misma parte de lo que se evalúa.

Response: Aquí tienes la implementación de la clase para generar imágenes BMP desde cero. Está escrita en Python porque es ideal para estructurar la lógica orientada a objetos, pero su arquitectura (crear matriz, asignar píxeles, calcular padding y escribir bytes) se puede traducir directamente a C++ o Java.

1. Clase GeneradorBMP

Un archivo BMP de 24 bits requiere un encabezado general (14 bytes), un encabezado de información (40 bytes) y la información de los píxeles (escrita en formato Azul, Verde, Rojo). Además, requiere un padding matemático: cada fila de píxeles debe ser un múltiplo de 4 bytes.

```
import struct

class GeneradorBMP:
    def __init__(self, ancho, alto, color_fondo=(255, 255, 255)):
        self.ancho = ancho
        self.alto = alto
        # Inicializar matriz bidimensional para los píxeles (x, y)
        self.pixeles = [[color_fondo for _ in range(ancho)] for _ in range(alto)]

    def dibujar_pixel(self, x, y, color):
        """Asigna un color RGB a unas coordenadas específicas."""
        # Prevenir errores de índices si se intenta dibujar fuera de la imagen
        if 0 <= x < self.ancho and 0 <= y < self.alto:
            self.pixeles[y][x] = color

    def dibujar_rectangulo(self, x_inicio, y_inicio, ancho, alto, color):
        """Método auxiliar para generar figuras de prueba."""
        for y in range(y_inicio, y_inicio + alto):
            for x in range(x_inicio, x_inicio + ancho):
                self.dibujar_pixel(x, y, color)

    def guardar(self, ruta_archivo):
        """Constuye los encabezados y guarda el archivo binario BMP."""
        bytes_por_fila = self.ancho * 3
        # Las filas en BMP deben ser múltiplos de 4 bytes
        padding = (4 - (bytes_por_fila % 4)) % 4
        tamano_datos = (bytes_por_fila + padding) * self.alto
        tamano_archivo = 54 + tamano_datos # 54 bytes fijos de los encabezados

        try:
            with open(ruta_archivo, 'wb') as f:
                # --- HEADER DEL ARCHIVO BMP (14 bytes) ---
                f.write(b'BM') # Firma
                f.write(struct.pack('<L', tamano_archivo)) # Tamaño total
                f.write(struct.pack('<H', 0)) # Reservado
                f.write(struct.pack('<H', 0)) # Reservado
                f.write(struct.pack('<L', 54)) # Offset a los píxeles

                # --- DIB HEADER (40 bytes) ---
                f.write(struct.pack('<L', 40)) # Tamaño del DIB header
                f.write(struct.pack('<L', self.ancho)) # Ancho
                f.write(struct.pack('<L', self.alto)) # Alto
                f.write(struct.pack('<H', 1)) # Planos
                f.write(struct.pack('<H', 24)) # Bits por píxel (RGB)
                f.write(struct.pack('<L', 0)) # Sin compresión
                f.write(struct.pack('<L', tamano_datos)) # Tamaño de la imagen
                f.write(struct.pack('<L', 2835)) # Resolución horizontal
                f.write(struct.pack('<L', 2835)) # Resolución vertical
                f.write(struct.pack('<L', 0)) # Colores en paleta
                f.write(struct.pack('<L', 0)) # Colores importantes

                # --- DATOS DE LA IMAGEN ---
                # BMP lee los píxeles de abajo hacia arriba
```

```

        for y in range(self.alto - 1, -1, -1):
            for x in range(self.ancha):
                r, g, b = self.pixeles[y][x]
                # Escribir en formato BGR (estándar de BMP)
                f.write(struct.pack('<BBB', b, g, r))

            # Agregar el padding nulo al final de cada fila
            f.write(b'\x00' * padding)

    print(f"Éxito: Imagen generada en '{ruta_archivo}'.")

except FileNotFoundError:
    print("Error: La ruta o el directorio especificado no existe.")
except PermissionError:
    print("Error: No hay permisos para escribir en la ruta proporcionada.")
except Exception as e:
    print(f"Error crítico: No se pudo generar la imagen. ({e})")

# Ejemplo de uso desde el Main:
# bmp = GeneradorBMP(800, 600, color_fondo=(200, 200, 200))
# bmp.dibujar_rectangulo(100, 100, 200, 150, color=(255, 0, 0)) # Rectángulo rojo
# bmp.guardar("figura_prueba.bmp")

```

2. Análisis del Problema: Requisitos de tu Solución

Basado en el texto de tu asignación, aquí tienes la clasificación explícita que debes incluir en tu reporte técnico.

Requisitos Funcionales (Lo que el programa debe *hacer*)

1. **Lectura de archivos:** Recibir como entrada la ruta de una imagen en formato compatible (ej. BMP).
2. **Identificación de formas:** Detectar y aislar cada figura presente en la imagen leída.
3. **Extracción de color:** Determinar el color predominante o exacto de cada figura detectada.
4. **Clasificación:** Asignar cada figura a una de las cuatro categorías predefinidas según sus características morfológicas.
5. **Generación de reporte:** Emitir un resultado en pantalla o archivo que enumere las figuras encontradas, sus colores y sus clasificaciones.

Requisitos No Funcionales (Cómo se debe *comportar* el programa)

1. **Robustez ante errores:** El sistema debe capturar entradas inválidas (rutas incorrectas, archivos corruptos, formatos no soportados) y mostrar mensajes de advertencia claros, evitando caídas abruptas (cierres inesperados o *crashes*).
2. **Rendimiento (Performance):** El algoritmo debe procesar y analizar imágenes de tamaño promedio en un tiempo razonable, gestionando eficientemente la memoria para evitar saturación de RAM.
3. **Mantenibilidad:** El código debe estar basado en el paradigma orientado a objetos (clases `LectorColor`, `ClasificadorFiguras`, `GeneradorBMP`), con alta cohesión y nombres descriptivos para facilitar revisiones futuras.
4. **Usabilidad:** Las instrucciones y resultados emitidos en la interfaz (ya sea de terminal o gráfica) deben ser intuitivos y legibles para cualquier usuario sin conocimientos técnicos.

User prompt: para la primera clase que genera imagenes, pude ser que genere las imagenes por separado, ya que necesito por lo menos 10 imagenes para probar el código

Response: ¡Claro que sí! Para generar las 10 imágenes por separado y que te sirvan para probar tus otras clases (el lector de color y el identificador de figuras), la mejor estrategia es **crear un pequeño programa o función de prueba que instancie la clase GeneradorBMP 10 veces**, dibujando figuras de distintos tamaños y colores en cada una.

Para que las pruebas sean útiles para tu clasificador, vamos a usar combinaciones de rectángulos para simular diferentes categorías (por ejemplo: Cuadrados, Rectángulos, Cruces y formas en "L").

Aquí tienes cómo hacer el *script* para generar automáticamente las 10 imágenes de prueba:

Código para generar 10 imágenes de prueba

Añade este código debajo de la clase `GeneradorBMP` que te compartí anteriormente.

```

import os

def generar_lote_de_pruebas():
    # Crear un directorio para guardar las imágenes si no existe
    if not os.path.exists("imagenes_prueba"):
        os.makedirs("imagenes_prueba")

```

```

# Lista de configuraciones para las 10 imágenes
# Cada elemento tiene: (nombre_archivo, color_fondo, tipo_figura, color_figura)
pruebas = [
    ("prueba_01_cuadrado.bmp", (255, 255, 255), "cuadrado", (255, 0, 0)),      # Rojo
    ("prueba_02_rectangulo.bmp", (255, 255, 255), "rectangulo", (0, 255, 0)),  # Verde
    ("prueba_03_cruz.bmp", (255, 255, 255), "cruz", (0, 0, 255)),              # Azul
    ("prueba_04_forma_L.bmp", (255, 255, 255), "L", (255, 255, 0)),            # Amarillo
    ("prueba_05_cuadrado.bmp", (0, 0, 0), "cuadrado", (0, 255, 255)),          # Cian (fondo negro)
    ("prueba_06_rectangulo.bmp", (200, 200, 200), "rectangulo", (255, 0, 255)), # Magenta
    ("prueba_07_cruz.bmp", (255, 255, 255), "cruz", (128, 0, 128)),            # Morado
    ("prueba_08_forma_L.bmp", (255, 255, 255), "L", (255, 165, 0)),            # Naranja
    ("prueba_09_cuadrado_pequeno.bmp", (255, 255, 255), "cuadrado_peq", (0, 0, 0)), # Negro
    ("prueba_10_rectangulo_largo.bmp", (255, 255, 255), "rectangulo_largo", (100, 100, 255))
]

ancho_img = 400
alto_img = 400

print("Generando lote de 10 imágenes de prueba...")

for nombre, color_fondo, forma, color_figura in pruebas:
    # 1. Crear una nueva instancia por cada imagen
    bmp = GeneradorBMP(ancho_img, alto_img, color_fondo)

    # 2. Dibujar la figura correspondiente usando rectángulos
    if forma == "cuadrado":
        bmp.dibujar_rectangulo(100, 100, 200, 200, color_figura)

    elif forma == "rectangulo":
        bmp.dibujar_rectangulo(50, 150, 300, 100, color_figura)

    elif forma == "cruz":
        # Una cruz hecha de dos rectángulos superpuestos
        bmp.dibujar_rectangulo(150, 50, 100, 300, color_figura) # Vertical
        bmp.dibujar_rectangulo(50, 150, 300, 100, color_figura) # Horizontal

    elif forma == "L":
        # Forma de L hecha con dos rectángulos
        bmp.dibujar_rectangulo(100, 100, 100, 200, color_figura) # Vertical
        bmp.dibujar_rectangulo(100, 200, 200, 100, color_figura) # Horizontal base

    elif forma == "cuadrado_peq":
        bmp.dibujar_rectangulo(180, 180, 40, 40, color_figura)

    elif forma == "rectangulo_largo":
        bmp.dibujar_rectangulo(20, 180, 360, 40, color_figura)

    # 3. Guardar el archivo en la carpeta
    ruta_final = os.path.join("imagenes_prueba", nombre)
    bmp.guardar(ruta_final)

# Llamar a la función para ejecutar todo
if __name__ == "__main__":
    generar_lote_de_pruebas()

```

¿Por qué hacerlo de esta manera te ayuda en tu proyecto?

- 1. Aislamiento de casos de prueba:** Al tener 10 imágenes distintas, cuando programes tu clase `IdentificadorFiguras`, podrás probar si tu lógica distingue correctamente entre un cuadrado y un rectángulo comparando la proporción entre el ancho y el alto.
- 2. Pruebas de la clase `LectorColor`:** Varié intencionalmente los colores (Rojo, Verde, Azul, Amarillo, Cian, Magenta, etc.) y un fondo negro en una de ellas. Esto te servirá para confirmar que tu lector ignora el color de fondo y detecta correctamente los valores RGB de la figura.
- 3. Automatización:** Si de repente decides que tus imágenes deben ser de 800x600 en lugar de 400x400, solo cambias los valores `ancho_img` y `alto_img` en este código, lo vuelves a correr y en menos de un segundo tienes 10 imágenes nuevas perfectamente generadas.

¿Quieres que te ayude a empezar con la lógica de la clase **LectorColor** o la de **Averiguar la Figura** usando los píxeles de estas imágenes que acabamos de generar?